

Unlocking Peak Performance

NVIDIA HPC Architecture and CUDA Optimization Techniques

ECE759 Guest Lecture

Matthew Bruehl
11-Apr-2025



A Few Disclaimers...

I'm not a CUDA expert, still learning!

Why does this matter?

- Companies want efficiency

Industry insights are helpful in academia

How do you prepare for your career?

Explore related technologies

Practice with AI

I'm not here for NVIDIA, but as an
alumnus

This is not a *formal* training session

I'm not a PhD or research expert

This is just a tease of what's possible

Learn outside the classroom

My Background



BS ECE 2000

Focus on Digital
Design/CE



2000-2005

Circuit Design
for Itanium FPU



2005-2022

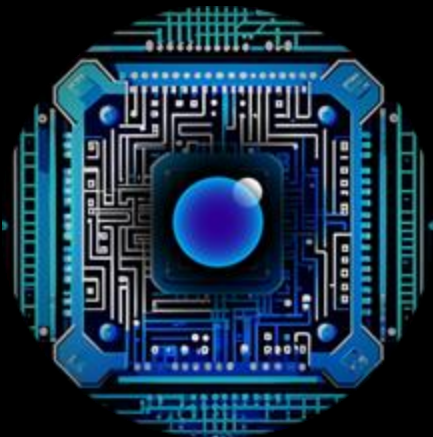
Chip
composition and
Data analytics



2023-Now

AI infrastructure
and inference
development

Outline



State of Arch:
CPU, GPU, TPU,
NPU



GPU Arch and
Blackwell

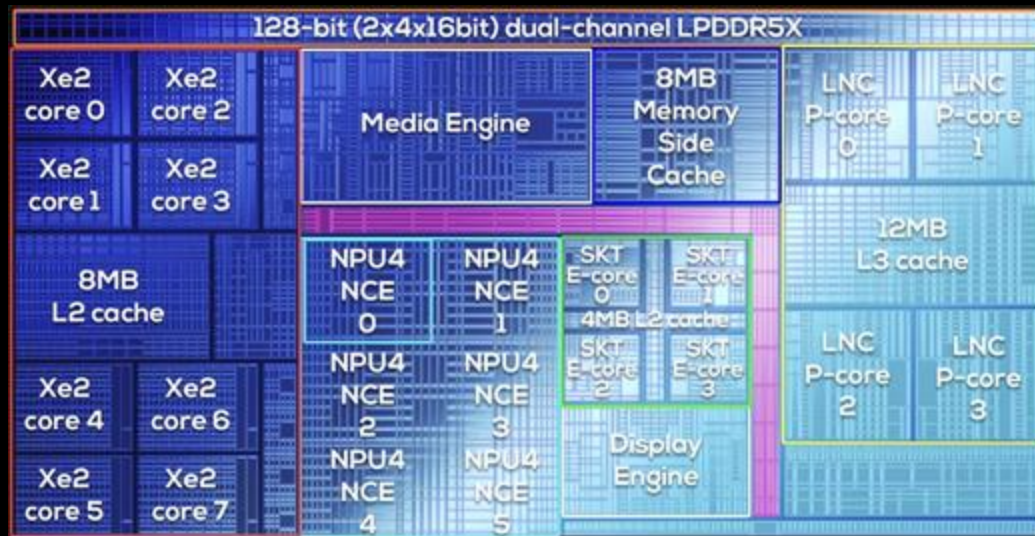


CUDA Tips

State of The Arch

NPU (Intel Lunar Lake): Dedicated for AI acceleration integrated into SoC

- Optimized for on device AI inference at low power; limited scalability; excels at real time AI inference at the edge



CPU (Intel Raptor Lake): General purpose, suitable for diverse tasks

- Limited parallelism for massive data processing; handle like AI workloads



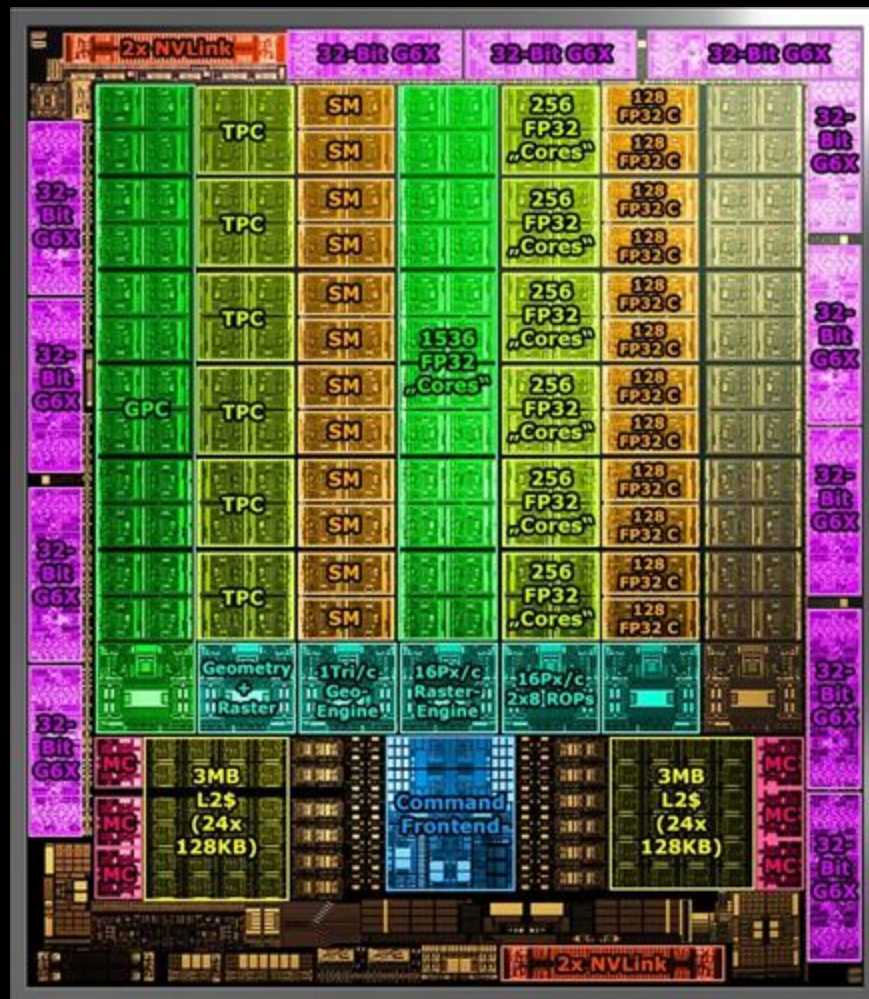
State of The Arch

GPU: (NVIDIA Blackwell): Parallel architecture with thousands of small cores for large scale computation

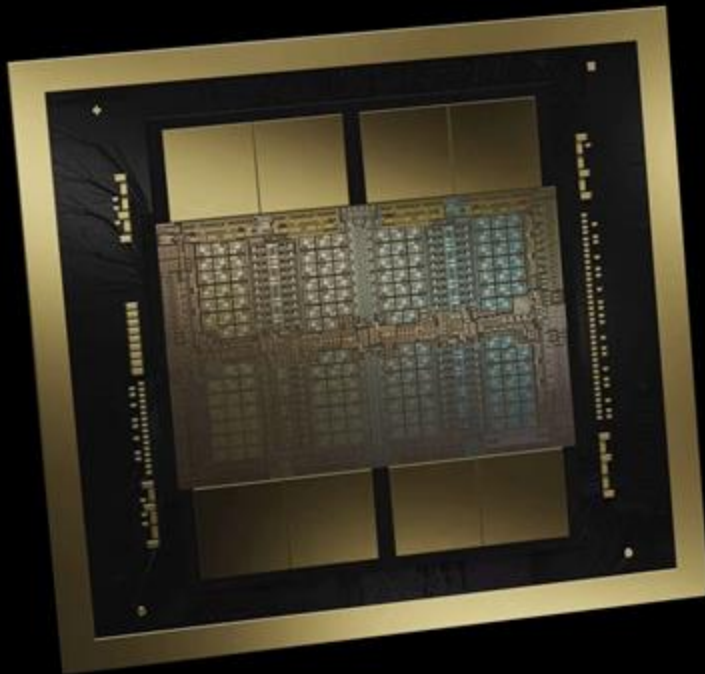
- High computation throughput; high power consumption; requires fine grained control; excels as model training and inference

TPU: (Google Ironwood): Extremely efficient for training and inference; limited flexibility; low latency

- Ironwood has an enhanced specialized core, SparseCore, for processing the types of data common in “advanced ranking” and “recommendation” workloads (e.g. an algorithm that suggests apparel you might like).



NVIDIA Blackwell GPU



BLACKWELL

THE ENGINE OF THE NEW INDUSTRIAL REVOLUTION

20 petaFLOPS of AI performance

192GB of HBM3e

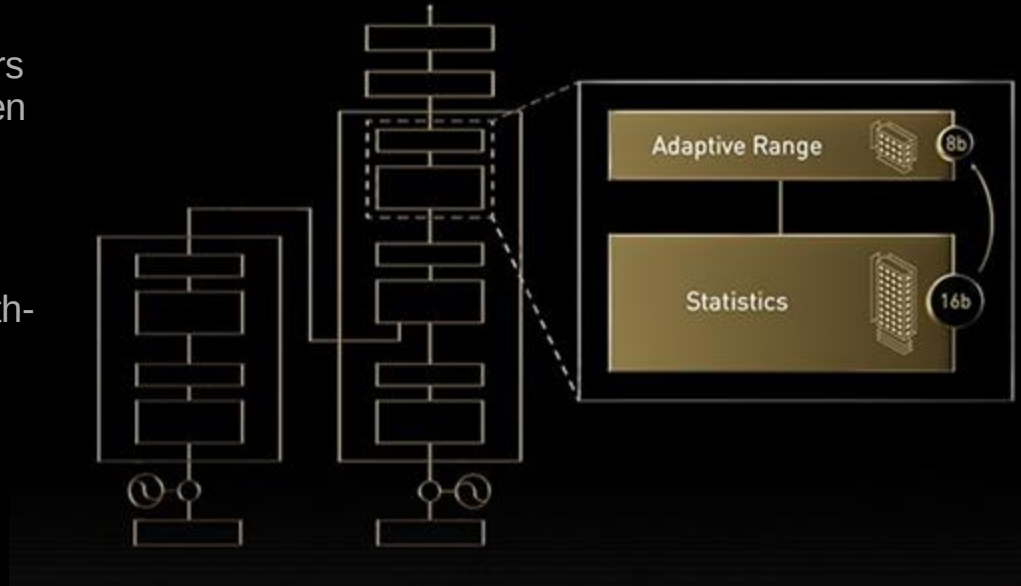
8TB/s of memory bandwidth

Full stack, CUDA enabled

NVIDIA Blackwell Key Features

Dual-Die Architecture: Blackwell features a cutting-edge dual-die GPU design, interconnected by NVIDIA's ultra-fast NV-HBI (High Bandwidth Interface). This design delivers over 10 TB/s of bidirectional bandwidth between dies, enabling what appears to software as a single, unified GPU—greatly boosting performance and scalability.

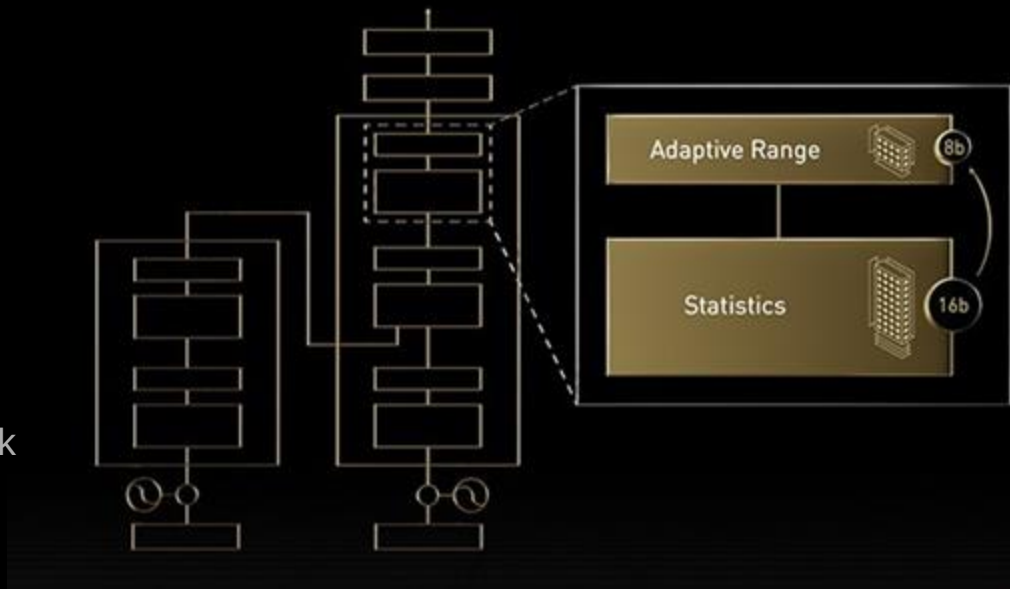
Next-Generation Tensor Cores: The new 5th-generation Tensor Cores introduce support for advanced data types like FP4 and FP6, in addition to existing formats (e.g., TF32, FP8, INT8). This enables higher throughput for AI inference and training, while improving energy efficiency and precision.



NVIDIA Blackwell Key Features

Transformer Engine Enhancements: The improved Transformer Engine includes dynamic range management and fine-grained precision control. This allows it to automatically choose the optimal precision (FP8, FP6, or FP4) for each layer or tensor, significantly accelerating LLM and GenAI workloads.

Memory and Interconnect Innovations: Blackwell supports up to 192 GB of ultra-fast HBM3E memory with over 8 TB/s bandwidth. Combined with NVLink Switch System interconnects (allowing up to 576 GPUs to work together), it dramatically enhances multi-GPU workload efficiency for massive AI models and scientific computing.



Blackwell GB202 GPU

The **full GB202 GPU** includes 12 Graphics Processing Clusters (GPCs), 96 Texture Processing Clusters (TPCs), 192 Streaming Multiprocessors (SMs), and a 512-bit memory interface with sixteen 32-bit memory controllers.



Figure 3. GB202 GPU block diagram (full chip).

Note: The GB202 GPU also includes 384 FP64 Cores (two per SM) which are not depicted in the above diagram. The FP64 TFLOP rate is 1/64th the TFLOP rate of FP32 operations. The small number of FP64 Cores are included to ensure any programs with FP64 code operate correctly. Similarly, a very minimal number of FP64 Tensor Cores are included for program correctness.

The full GB202 GPU includes:

- 24576 CUDA Cores
- 192 RT Cores
- 768 Tensor Cores
- 768 Texture Units

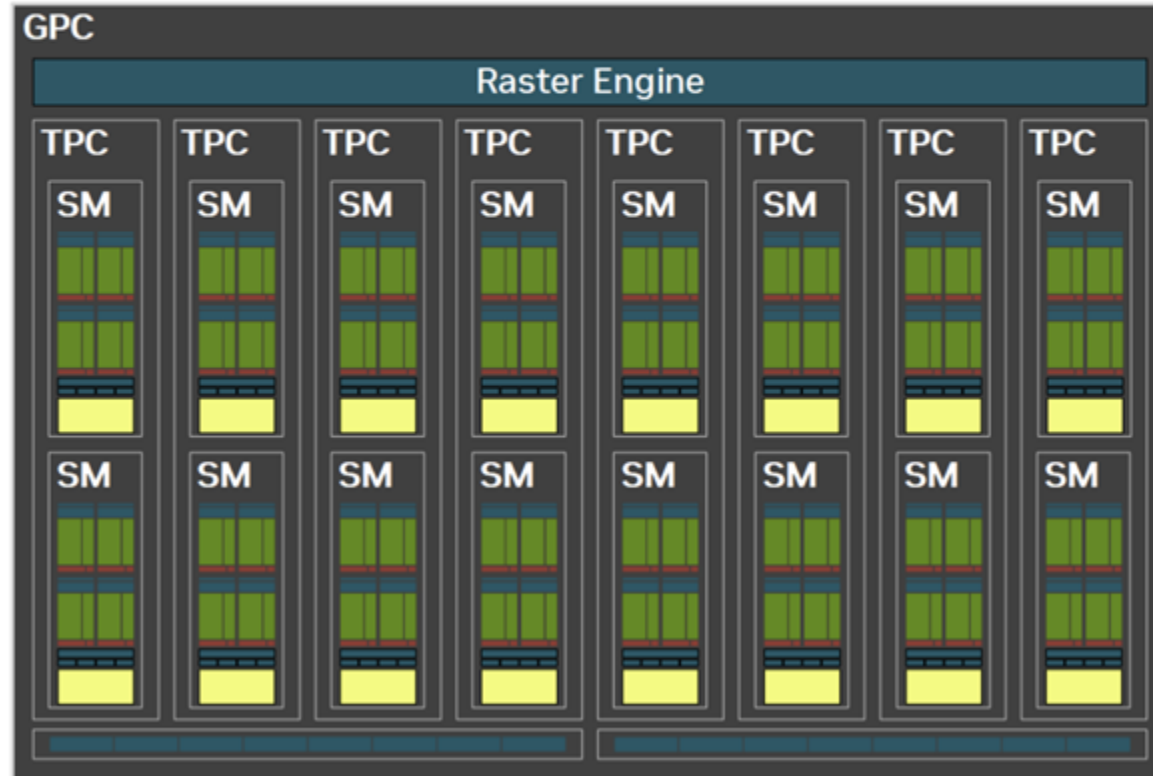


Figure 4. The Blackwell GPC with Raster Engine, 8 TPCs, 16 SMs, and 16 ROPs.

NVIDIA B200 Streaming Multiprocessor (SM)

The NVIDIA Streaming Multiprocessor (SM) is a core component of the NVIDIA GPU architecture, playing a key role in the parallel processing capabilities of the GPU, enabling massive parallelism through its various cores (CUDA, Tensor, RT), efficient warp scheduling, memory management, and support for modern workloads like AI.

Each full GB202 chip contains 192 SMs, and each SM includes 128 CUDA Cores, one Blackwell Fourth-Generation RT Core, four Blackwell Fifth Generation Tensor Cores, 4 Texture Units, a 256 KB Register File, and 128 KB of L1/Shared Memory, which can be configured for different memory sizes depending on the needs of the graphics and compute workloads.



Programming Model Enhancements

1. Unified Memory Improvements

- Blackwell's enhanced memory subsystem will lead to improvements in CUDA's Unified Memory model resulting in more efficient automatic memory management and reduced programmer overhead for large datasets.

2. Enhanced Tensor Core Programming

- The second-generation Transformer Engine in Blackwell supports new 4-bit floating point AI inference capabilities. This will likely lead to new CUDA libraries and programming models optimized for these low-precision operations, particularly beneficial for AI-driven HPC applications.

3. Quantum-Classical Hybrid Computing

- NVIDIA's CUDA Quantum platform (formerly CUDA-Q) is gaining traction for quantum computing research. We can expect more tools and libraries for seamlessly integrating classical GPU computations with quantum algorithms, opening new avenues for HPC developers.

4. AI-Assisted HPC Programming

- With the increased focus on AI, we might see the development of AI-assisted coding tools specifically for HPC applications. These could help in optimizing CUDA kernels, suggesting optimal memory access patterns, or even generating entire HPC algorithms based on high-level descriptions.

CUDA Tips





[Advanced Strategies for High-Performance GPU Programming with NVIDIA CUDA | NVIDIA Technical Blog](https://developer.nvidia.com/blog/advanced-strategies-for-high-performance-gpu-programming-with-nvidia-cuda/)

<https://developer.nvidia.com/blog/advanced-strategies-for-high-performance-gpu-programming-with-nvidia-cuda/>

Advanced Parallelism Techniques in AI Inference for Trillion-Parameter Models

The deployment of trillion-parameter models like the GPT 1.8T MoE (Mixture of Experts) presents unique challenges in AI inference, particularly in managing computational resources effectively while ensuring optimal user experience.

Data Parallelism (DP)

Data parallelism involves hosting multiple copies of the entire model across different GPUs or clusters, processing independent user requests simultaneously.

This approach scales linearly with the number of GPUs, enhancing throughput without impacting user interactivity.

However, it requires significant memory as each GPU holds a complete model copy.

Tensor Parallelism (TP)

Tensor parallelism splits each layer of the model across multiple GPUs, allowing different parts of a user request to be processed in parallel.

This method can improve user interactivity by allocating more resources per request, thus reducing processing time.

However, it relies heavily on high-bandwidth inter-GPU communication, which can become a bottleneck at large scales.

Pipeline Parallelism (PP)

In pipeline parallelism, different groups of model layers are distributed across GPUs, with each part of a user request processed sequentially across the pipeline.

This technique helps manage large models by distributing weights, but may lead to inefficiencies in processing and does not significantly enhance user interactivity.

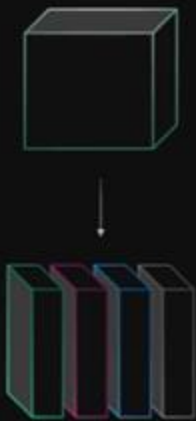
Expert Parallelism (EP)

Expert parallelism routes requests to specific experts within the model to different GPUs, reducing the interaction with unnecessary parameters.

The results, after expert processing, require all-to-all communication over high bandwidth GPU interconnect.

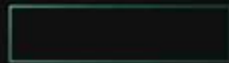
It requires complex management of data routing and reassembly, and its effectiveness is limited by the number of available experts.

There Are Many Types of Parallelism Patterns



Tensor Parallelism

Divide data along **dimensions** across processors



Data Parallelism

Divide individual **elements** across processors



Task Parallelism

Divide independent **workflows** across processors



Model Parallelism

Divide sequences of **operations** across processors

Focus on The Parallels

Fundamentally there are two parallelism types:

- How you break data down across threads
- How you break work into separate pieces

CUDA is intrinsically both (task and data parallel)

Essentially, Tensor parallelism is a form of Data parallelism

When coding in CUDA, think about how GPU maps work onto threads and how you can exploit that

Think of CPU as a latency machine, GPU as throughput machine

Combining Parallelism Techniques

Combining different parallelism methods can mitigate the limitations of individual techniques.

Using both expert and pipeline parallelism can double user interactivity with minimal loss in throughput.

Similarly, integrating tensor, expert, and pipeline parallelism can triple GPU throughput without sacrificing user interactivity.

Combining different parallelisms for the right deployment scenario is an exhaustive solution space exploration and requires a large set of compute resources.

Optimizing Data Parallelism

1. Single-Wave Kernels:

- Map data to threads rather than threads to data.
- Aim for kernel launches that fill the GPU in a single wave.
- This approach minimizes wave quantization effects, which can waste up to 50% of GPU resources on the final partial wave.

2. Grid-Stride Loops:

- Similar to single-wave kernels, but allows for more flexibility with different GPU sizes.
- Each thread processes multiple elements in a strided fashion.

3. Load Balancing:

- For cases where perfect single-wave kernels aren't possible, alternate tile sizes (e.g., 11 and 12) to balance workload across SMs.

4. Avoiding Row-Major Access Patterns:

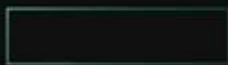
- Alternate block indexing between forward and backward to maximize cache hit rates.
- Example: ``int blockIdx = gridDim.x - blockIdx.x - 1;``

CUDA is Both: Data Parallelism Inside Task Parallelism



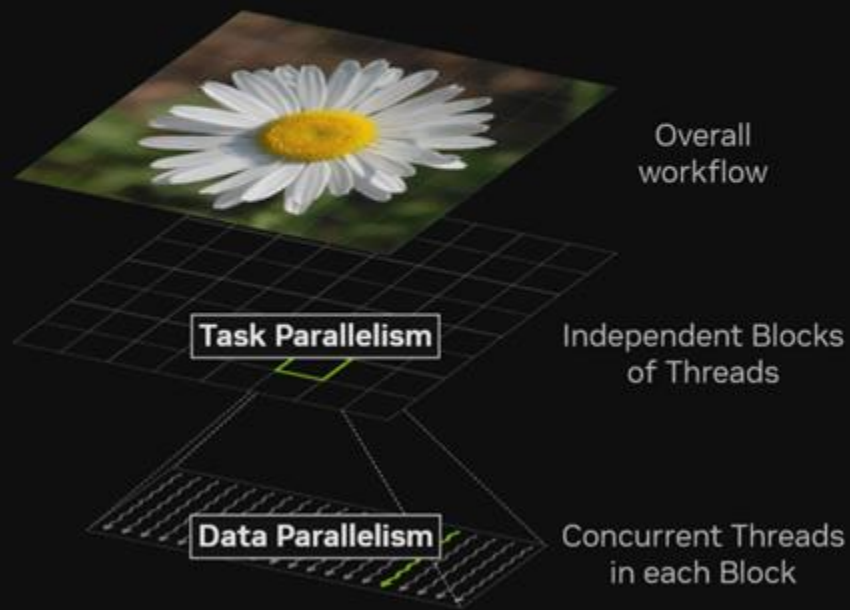
Task Parallelism

Divide independent **workflows** across processors



Data Parallelism

Divide individual **elements** across processors



Optimizing Data Parallelism

When the GPU launches a grid of threads for a kernel, that grid is divided into waves of thread blocks.

The size of a wave depends on the number of SMs on the GPU and the **Theoretical Occupancy** of the kernel.

On a NVIDIA Tesla K20 there are 13 SMs and the Theoretical Occupancy of a kernel was 4 blocks of 256 threads per SM (50%). Each full wave was composed by $13 \times 4 = 52$ blocks.

Since the kernel is only launching a total of 128 blocks (a very low number), the code is executed in 2 full waves and a much smaller wave of 24 blocks.

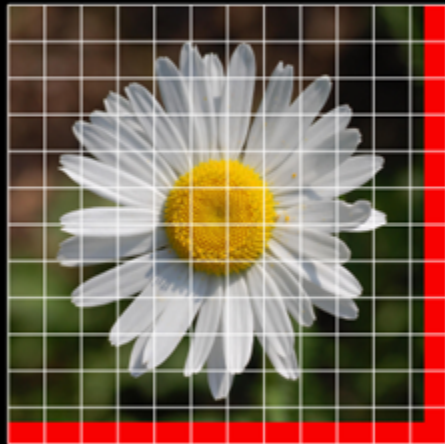
The last wave under-utilizes the GPU but represents a significant fraction of the run time.

To improve the performance of the kernel, use the `__launch_bounds__` attribute to constrain the number of registers since it is the main occupancy limiter.

As a result, the kernel is now able to run 5 blocks of 256 threads per SM instead of 4 blocks as before. The same computation is achieved in 1 full wave of $13 \times 5 = 65$ blocks and an almost-full wave of 63 blocks.

Ninjas Use Single-Wave Kernels

Don't map threads to data; map data to threads



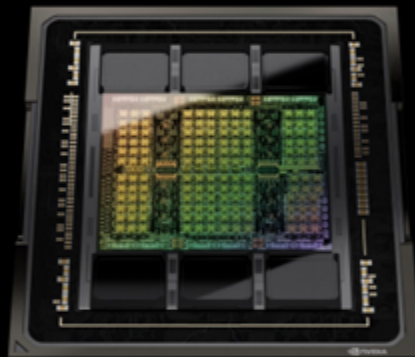
1024x1024 image divided into **12x12 tiles**

Red edge indicates where naïve rounding leaves imbalanced workload(*)

Image suggests 16x16 tiles = 256 blocks

Hardware suggests $\sqrt{132}$ SMs = 11.5 x 11.5 tiles

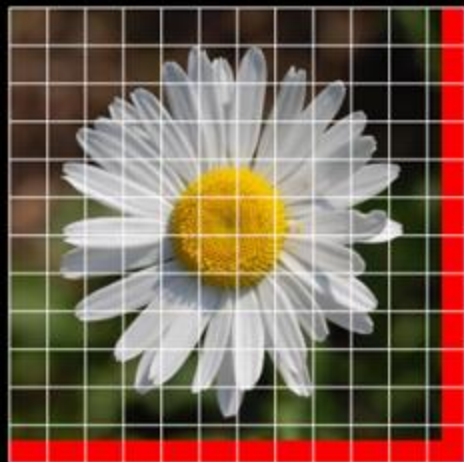
Where possible, always map data to threads



H100 with 132 SMs

(*) For better load balancing alternate tile sizes of 11 and 12, although this increases complexity and may be impossible depending on application

Not a Silver Bullet



Single-wave kernels are better in almost all cases than non-integer-wave

Similar to “grid-stride loop” pattern frequently taught for CUDA

But there are a number of problems which may prevent use:

1. Some algorithms require specific size of tiling
2. Must account for GPUs of different sizes (e.g. RTX-3090/80/70/60)
3. Increase in code complexity by having non-constant tile size
4. Load imbalance remains; may be no better than an extra partial wave

Optimizing Task Parallelism

1. CUDA Streams:

- Use multiple streams to execute independent tasks concurrently.
- Create dependencies between streams using events and waits.

2. CUDA Graphs:

- Convert stream-based workflows into CUDA graphs for optimized execution.
- Graphs can represent complex dependencies and allow for better scheduling.

3. Minimizing Inter-Task Dependencies:

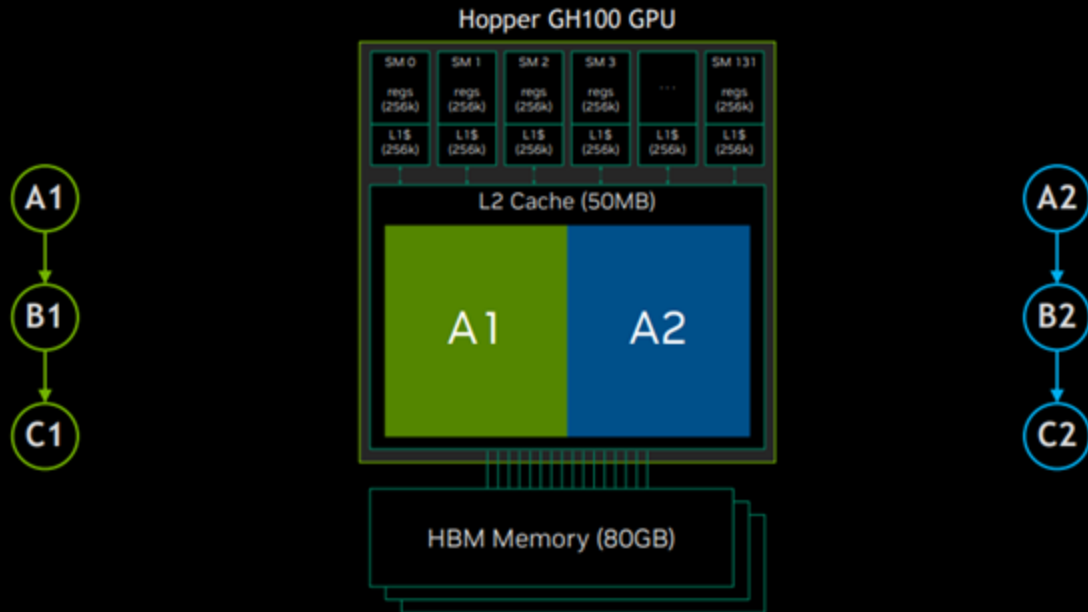
- When splitting complex workflows (e.g., in model parallelism), minimize dependencies between tasks.
- Unbalanced tasks are often cheaper than over-synchronization.

4. Hierarchical Splitting:

- For very large models or complex workflows, split tasks hierarchically to maintain efficiency.

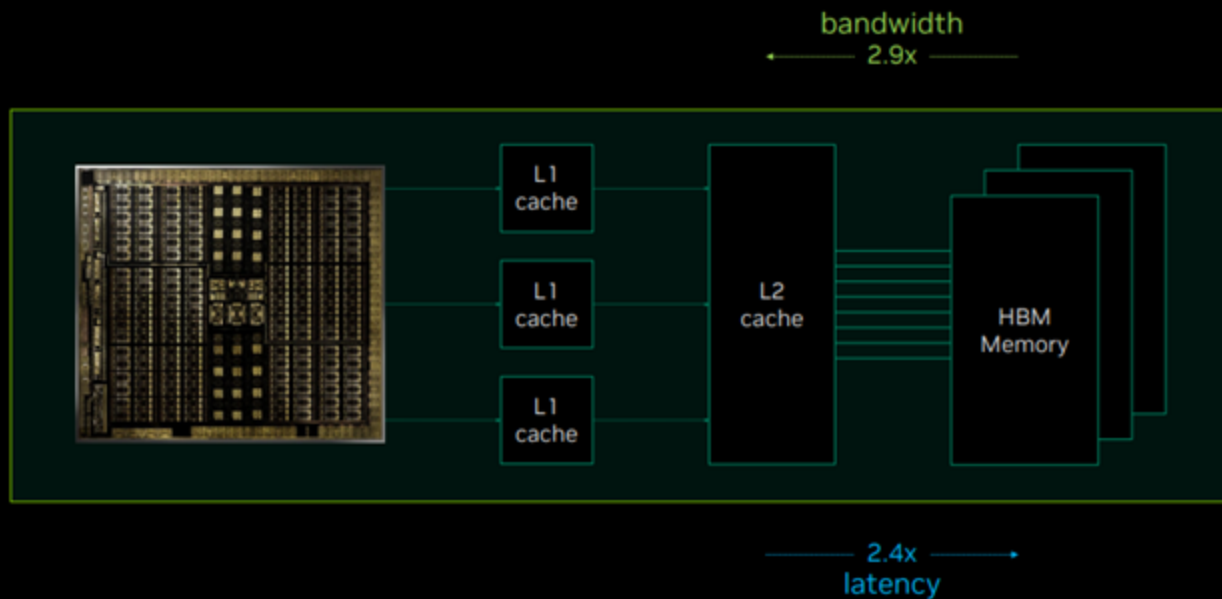
A Big Problem With Task Parallelism: Thrashing The Cache

Each task sees a proportionately smaller cache because by definition they are working on different data



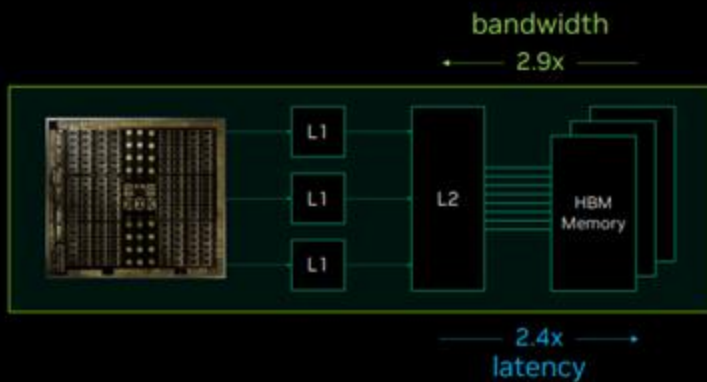
You really really want cached data

A high L2 hit rate can give as much as a 10x boost in performance

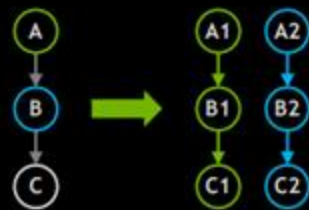




If most programs are limited by memory bandwidth ...



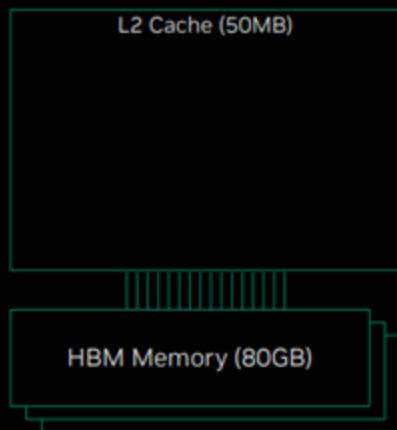
... and L2 cache can run as much as 10x faster than main memory ...



...and I know how to split my program into smaller tasks...

What If My Tasks Were Made Small Enough to Fit Entirely in L2 Cache?

Not all programs can do this, but more can than you might think...

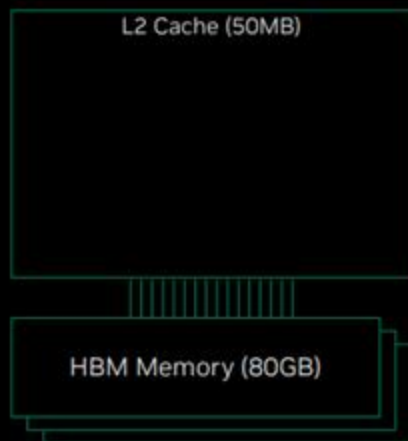


What If My Tasks Were Made Small Enough to Fit Entirely in L2 Cache?

Not all programs can do this, but more can than you might think...



Split data into cache-sized chunks

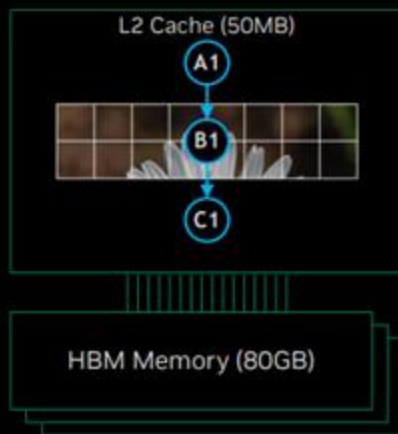


We'll have one task per chunk

To Keep Data in Cache We Run Each Task in Series, **NOT** in Parallel



Split data into cache-sized chunks

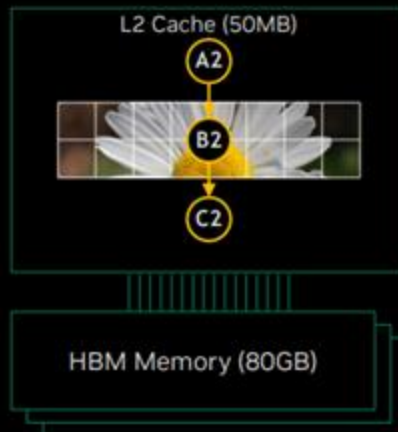


We'll have one task per chunk

To Keep Data in Cache We Run Each Task in Series, **NOT** in Parallel



Split data into cache-sized chunks

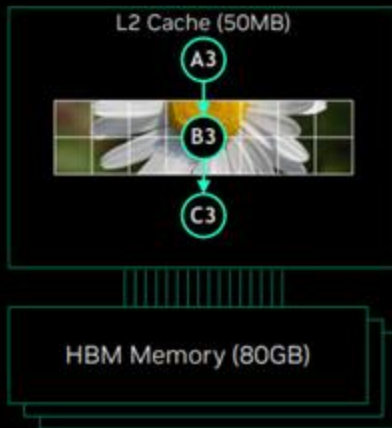


We'll have one task per chunk

To Keep Data in Cache We Run Each Task in Series, **NOT** in Parallel



Split data into cache-sized chunks

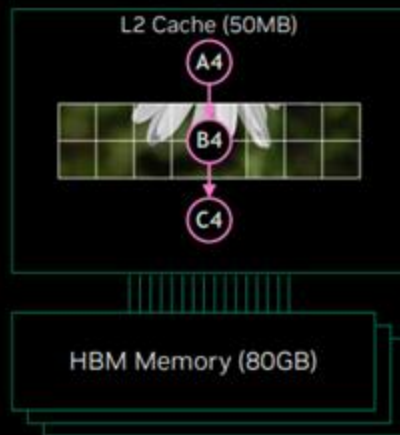


We'll have one task per chunk

To Keep Data in Cache We Run Each Task in Series, **NOT** in Parallel



Split data into cache-sized chunks



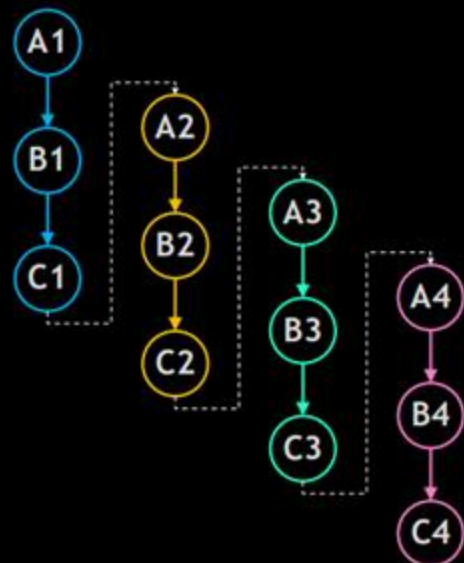
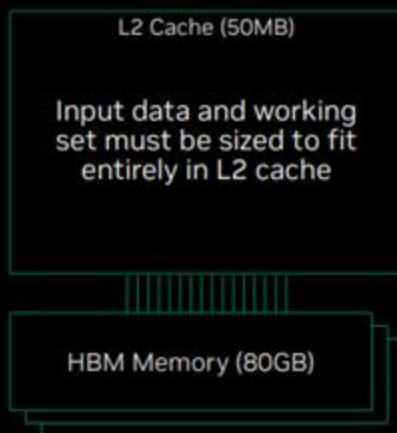
We'll have one task per chunk

This is Known As “Tiling” Your Execution in Cache

You'll really want to design your program for this up-front



Split data into cache-sized chunks



Run tasks in series

AI in Tech

According to Brookings Institution research, **the more advanced a tech worker's skills, the more vulnerable they may be to having their work supplemented or altered by generative AI technologies**

GenAI tools are specifically targeting cognitive tasks such as **coding, data analysis, and technical writing** – precisely the work performed by highly educated professionals in the tech sector

By 2027, Gartner Research projects that **GenAI tools assisting with code creation, testing, and operations will be adopted by approximately half of all enterprise software engineers**

AI Engineers are among the most sought-after specialists, commanding a median salary of \$106,386, with top earners reaching \$156,000. **These professionals design, develop, and implement AI systems and tools, serving as the backbone of organizations' AI capabilities**

As routine technical tasks become automated, the nature of entry-level positions is changing. **New professionals are increasingly expected to demonstrate AI literacy as a core competency from day one.** This includes understanding how to:

- Effectively interact with AI-generated outputs
- Verify the accuracy of AI-produced content
- Integrate AI tools into workflows to enhance productivity
- Provide the human oversight and judgment that AI systems lack

For You

Begin Preparing Now

For students preparing to enter the field, developing a combination of technical proficiency, AI literacy, and uniquely human capabilities will be essential for thriving in this transformed landscape.

The most successful professionals will be those who view AI as a powerful collaborator rather than a competitor, **leveraging these tools to enhance their own capabilities** while focusing their efforts on the tasks and decisions where human judgment, creativity, and ethical reasoning remain irreplaceable.

For You

Develop AI Literacy Alongside Core Technical Skills

While foundational technical competencies remain essential, students should actively develop AI literacy.

This includes understanding how to **effectively leverage AI tools in their specific domain**, **recognizing the limitations of current AI systems**, and identifying opportunities where human expertise and AI capabilities can complement each other.



Why Learn AI Now?

AI is impacting everyone and every workflow, right now.

AI Users, Adopters, Developers will flourish. Everyone else will languish*.

People in established careers are struggling to catch-up with the pace of change.

Fear, Uncertainty, Doubt reign supreme. Embrace it, and you may succeed.

Companies are desperate for candidates with AI integration experience (not just people who can type into ChatGPT)!

*not everyone, I'm exaggerating

Lessons Learned and Career Advice:

- Importance of versatility:
 - Example: Transitioning smoothly from physical hardware roles to software and AI infrastructure, showcasing **adaptability**.
- Communication and collaboration:
 - Example: Coordinating across global teams to solve cross-site design challenges, demonstrating **clear and effective communication skills**.
- Staying current and relevant in skills/knowledge:
 - Example: Regularly **upskilling or teaching** new technologies (like Kubernetes, Docker, ML) to hundreds of engineers, illustrating commitment to continuous education.
- Proactive innovation:
 - Example: Initiating and developing ChipNemo chatbot, demonstrating the **willingness to innovate** beyond established responsibilities

What About You?

- How can I effectively transition from academic projects to industry projects?
 - Start with structured training provided by the company, then seek mentorship to understand professional standards and expectations of your manager and team.
 - Take the time to learn and understand the impact of your project and your contributions.
- How important is it to specialize early versus gaining broad experience across different technologies?
 - Early breadth helps adapt quickly; specialization can come later as interests become clearer.

What About You?

- What's the best way to integrate into established teams and make meaningful contributions?
 - Listen first, learn team dynamics, then proactively offer help on smaller tasks to build trust.
 - Offer to contribute to peripheral projects and tasks, but don't get overwhelmed
- What are the essential skills for leadership roles?
 - Clear communication, technical expertise, empathy, team management, strategic thinking.
 - Example: Clearly articulate your ideas, provide data-driven rationale, and actively listen to colleagues to build consensus and persuade effectively.

IMHO: What Does Success Look Like?

People who are engaging, good listeners, and thoughtful communicators

People who take initiative and DELIVER RESULTS

You are measured by results, not the number of emails you send

People who adapt and adopt to changes and new technologies

Those who embrace learning will excel

People who take calculated risks AND learn from the outcomes

A Real Typical Day

Get Coffee first (most important task of day)

Scan Slack messages for anything important or exciting, or alerts from my automated service monitors

Check email (catchup from workers in other Geos) and new bug reports - reply if urgent, otherwise ignore

Check indicator dashboards, monitors, infrastructure metrics for anything unusual

Embark on continuing task work: researching, testing, experimenting, debugging, fixing

Task switch to other things when I get burned out, or attend some meetings



Careers

Senior Software Engineer - HPC JR1979406

We are looking for a Senior Software Engineer to join our mission to continue improving our HPC infrastructure. Our team builds and operates sophisticated infrastructure to enable business critical services and AI applications. You will be working with a team of passionate and skilled engineers that are continuously working to provide better tools to build and manage this infrastructure. Ideal candidate is strong in software development, designing and creating reliable distributed systems, and has the ability to implement well thought out long term maintenance strategy.

What you'll be doing:

- Design highly available and scalable systems to meet the demands of our HPC clusters
- Evaluate new and innovative technologies as the landscape evolves
- Continuously improve infrastructure provisioning and management using automation
- Support a globally distributed, multi-cloud hybrid environment - AWS, GCP and On-prem
- Build strong cross functional relationships and align with partners across various business units
- Ensure the highest level of up-time and Quality of Service (QoS) to our users through operational excellence
- Participate in team's on-call rotation and be a contact for service incidents

What we need to see:

- 10+ years of experience in design, implementation, and delivery of large engineering projects
- Comfortable with at least two of the following programming languages: Golang, Java, C/C++, Scala, Python, Elixir.
- Understands scalability challenges and performance of server-side code. Able to craft and develop horizontally-scalable, resilient and performing-under-load systems.
- Versatile technologist with experience in full software development lifecycle – from inception and design to deployment, operation, and iterative development.
- Proficient in cloud computing and are hands-on in at least one cloud platform: GCP, AWS, or Azure.
- Proficient in modern CI/CD techniques, GitOps and Infrastructure as Code(IaC)
- Strong work ethic and a passion for problem solving
- B.S. degree in Computer Science or related technical field (or equivalent experience)
- Detail oriented with great communication and collaboration skills

Ways to stand out from the crowd:

- Prior experience building solutions for HPC clusters based on Slurm or Kubernetes
- Strong understanding of Linux operation system and TCP/IP fundamentals

Leveraging AI

Use AI to review and offer shorter summaries, more impactful phrasing (action words) and have it ask you for numerical details around results and impacts

Have AI suggest formatting options and provide some examples

Write a prompt that asks AI to 'use the following job description', and customize my resume to match the desired skills and experience

Your task is to help me secure more job interviews by optimizing my resume to highlight relevant skills and experiences. You will perform this task in a series of steps.

<step1>Start by compiling all the current information from my resume. Gather details about my education, work experience, skills, certifications, and any relevant projects or volunteer work. Organize this information into categories for easy access and reference. Save this organized information in variable \$resume_data.</step1>

<step2>Using \$resume_data, write a new resume. Begin with a clear header that includes my name, contact information, and professional title or area of expertise. Structure the content into sections: Professional Summary, Skills, Work Experience, Education, and Additional Information. Focus on clarity and relevance to my career goals.</step2>

<step3>Modify my resume for each job application. Examine the job description carefully and identify keywords and skills that the employer emphasizes. Adjust my resume to highlight my qualifications that match these requirements. This customization shows employers my candidacy aligns well with the job expectations. Save the tailored resume in \$custom_resume.</step3>

<step4>Proofread my tailored resume for any errors in spelling, grammar, or formatting. Pay special attention to consistency in style and detail. Consider asking a friend or using professional services to review my resume. This ensures it is polished and professional. Store the final version of my resume in \$final_resume.</step4>

Begin by asking for to upload/paste the current resume and the type of job roles we're applying for.

Contact



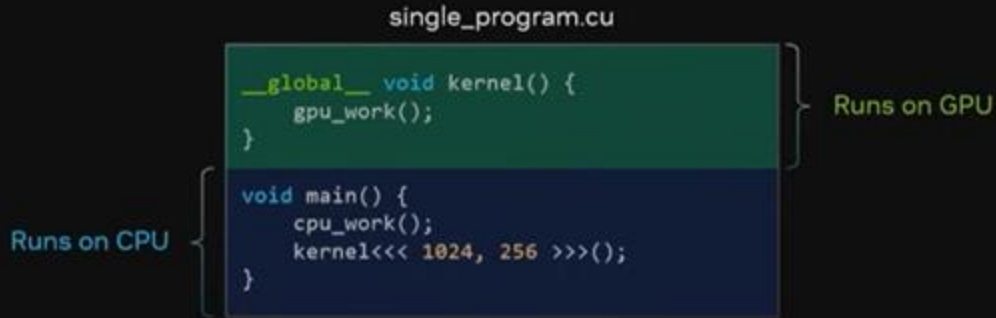
<https://www.linkedin.com/in/matt-bruehl/>



Thank You

The programming paradigm has always been:

- A GPU function annotated by global and a CPU function - one program running in two places



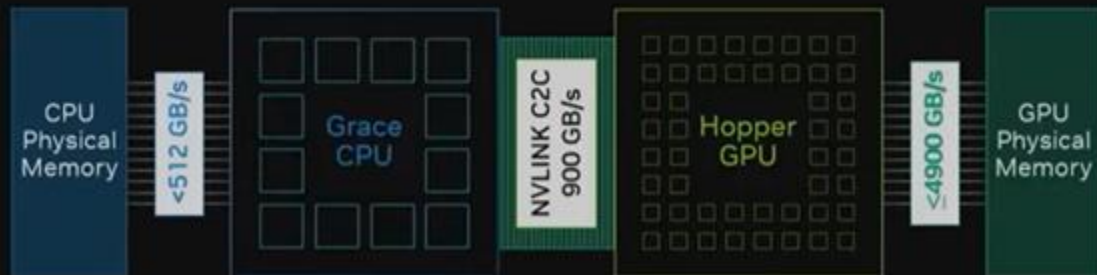
*Our key realization was that parallel computing
extends regular computing, not replaces it*

- NVIDIA CEO Jensen Huang

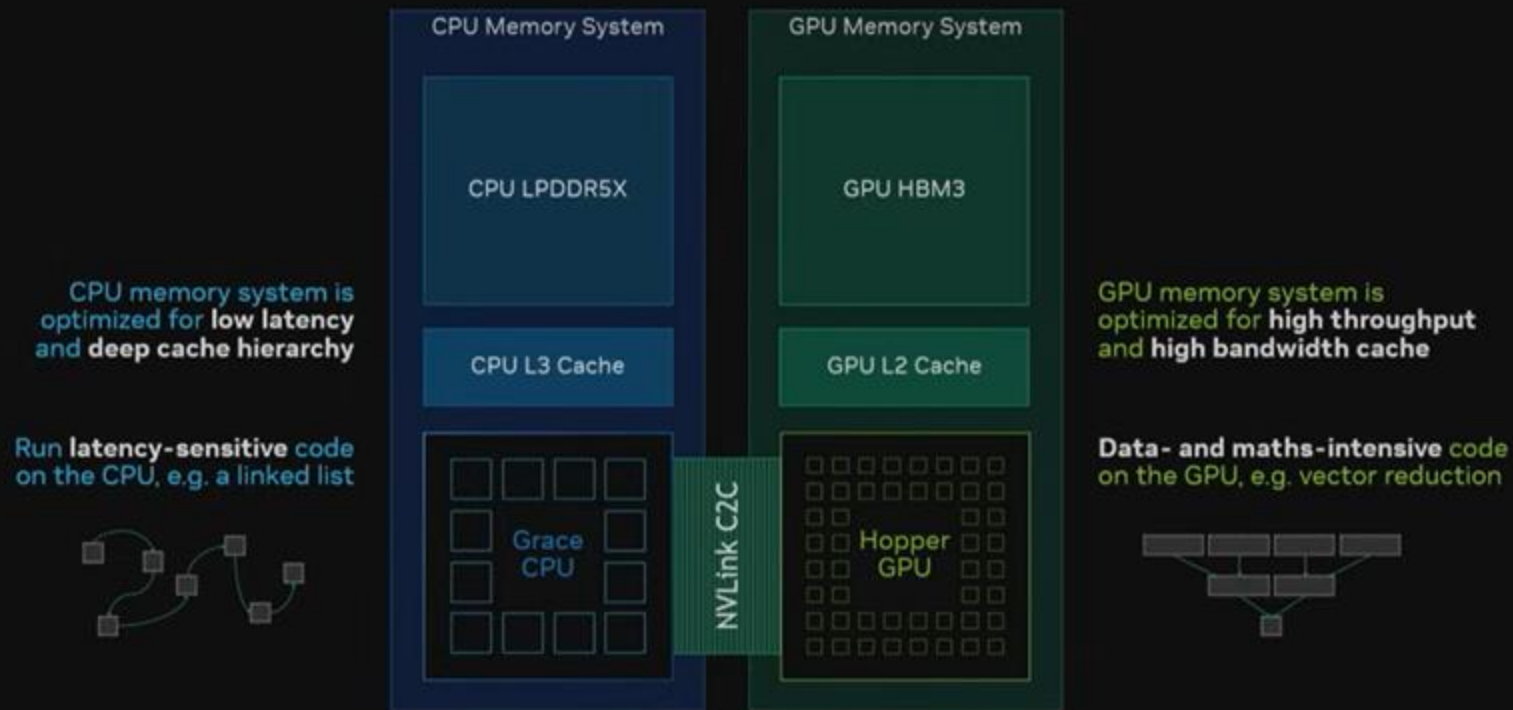
Grace-Hopper: A Unified Single Heterogeneous Processor

single_program.cu

```
__global__ void kernel() {  
    gpu_work();  
}  
  
void main() {  
    cpu_work();  
    kernel<<< 1024, 256 >>>();  
}
```

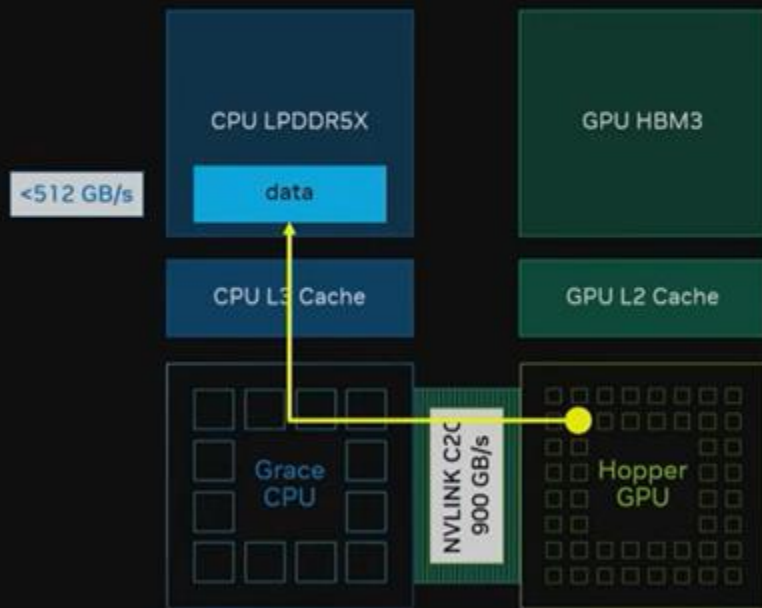


Two Memory Systems, Each Optimized For Its Processor



Two Memory Systems With One Address Space

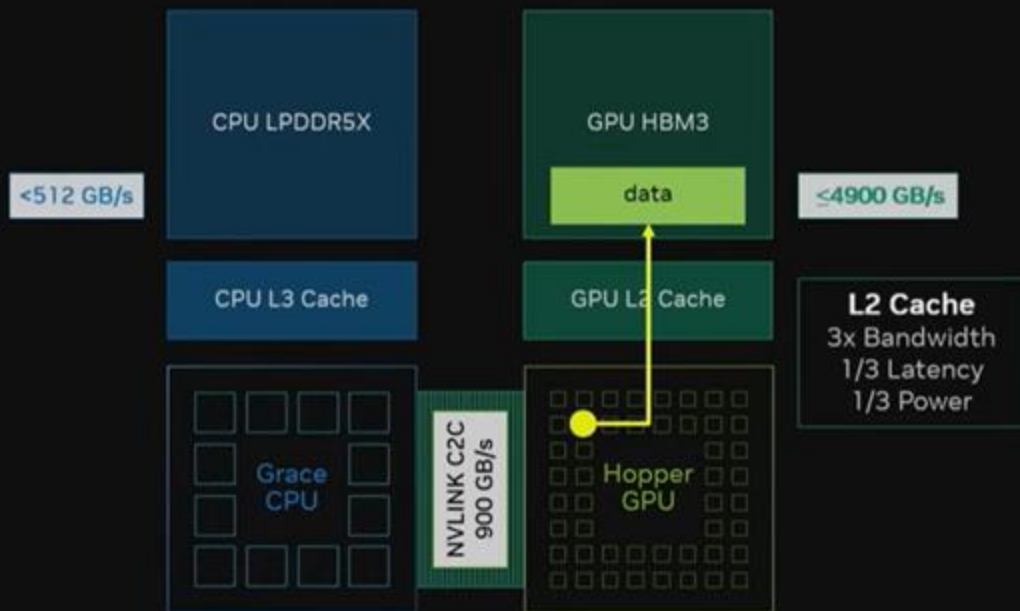
Both CPU and GPU can access the other's memory at any time



C2C connection is fast enough that GPU reads data at full LPDDR5X rate

Data Migration Optimizes GPU Accesses on GPU Memory System

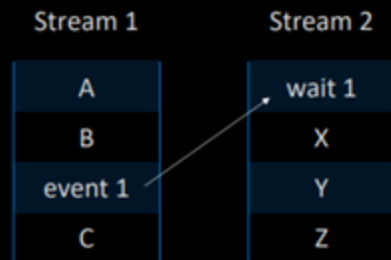
Data migration to the GPU can enable as much as 10x faster data access from the GPU



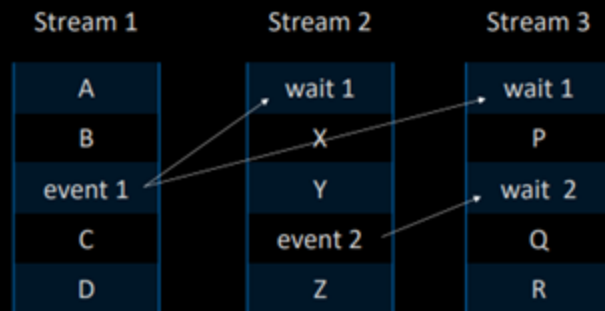
CUDA Makes Task Parallelism Easy To Do With Streams

Stream 1	Stream 2
A	X
B	Y
C	Z

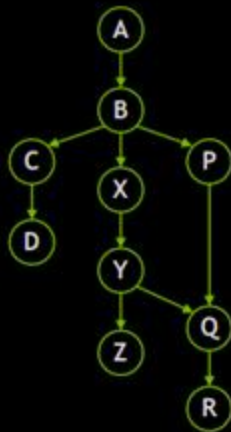
I Can Create Dependencies Between Streams



I Can Create Lots Of Streams With Lots Of Dependencies



I Can Turn It All Into A CUDA Graph

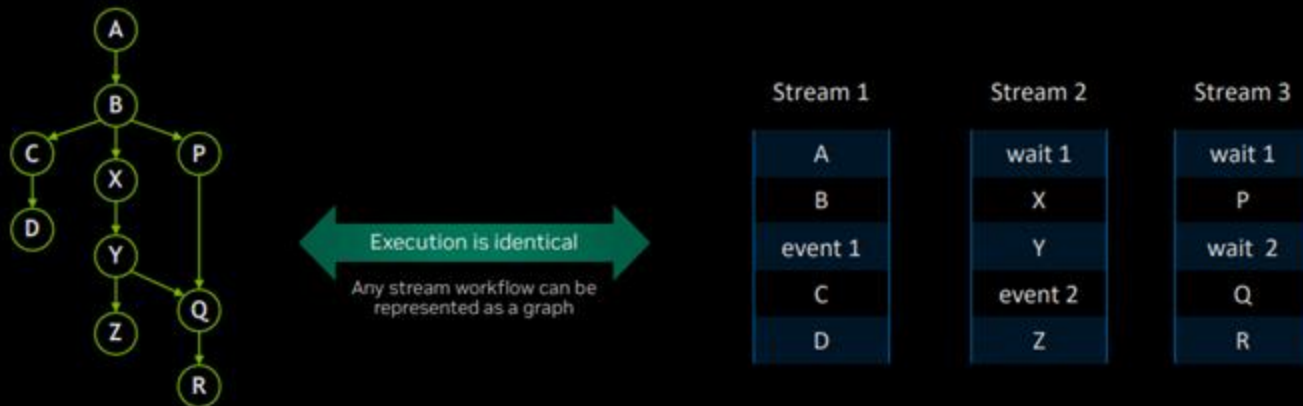


Execution is identical

Any stream workflow can be represented as a graph

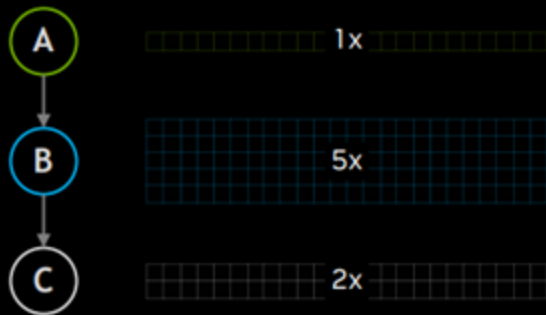
Stream 1	Stream 2	Stream 3
A	wait 1	wait 1
B	X	P
event 1	Y	wait 2
C	event 2	Q
D	Z	R

I Can Turn It Into A CUDA Graph



The problem is that to use these techniques I need lots of **independent** work in the first place

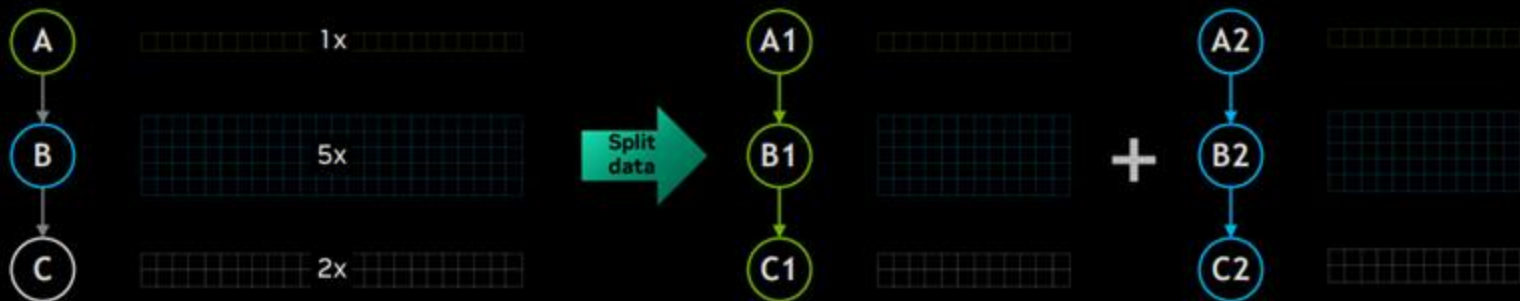
But What If I **DON'T** Conveniently Have Independent Work To Do?



How do I create task parallelism when there's only one task?

The Obvious Approach: Split The Data In Two

Also known as "Pipeline Parallelism" – yet another type of parallelism which is really just task parallelism

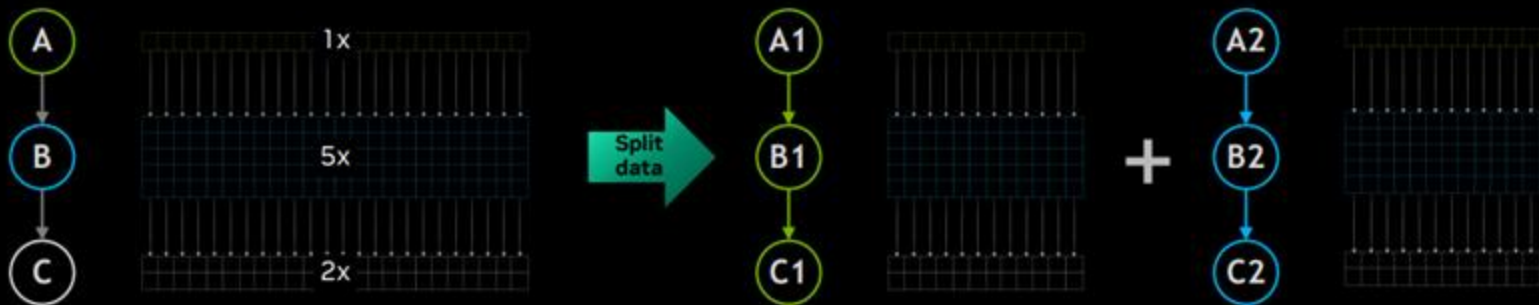


This Works Well If All Data In A1-B1-C1 Is Independent Of A2-B2-C2



Easy For Elementwise Programs

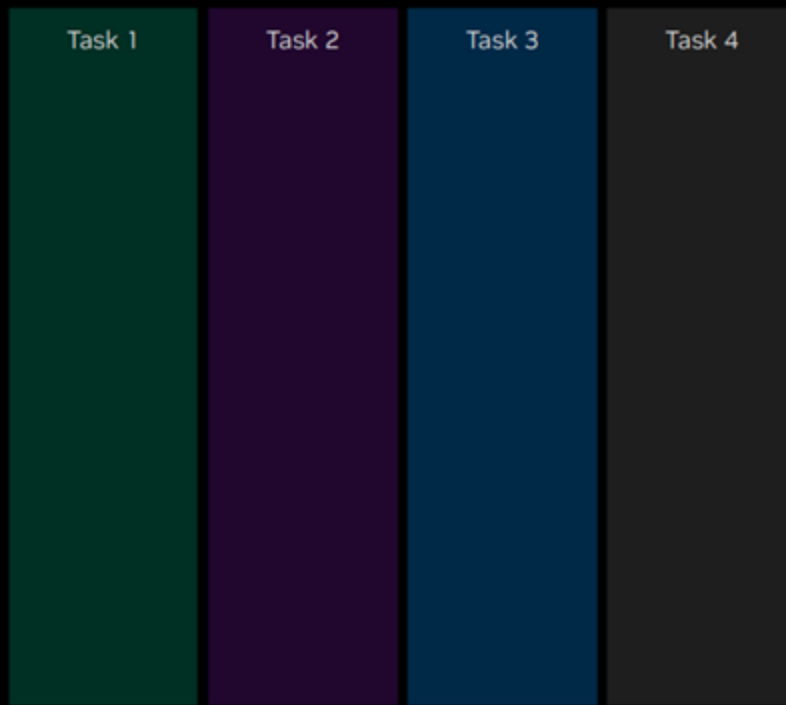
Programs never are entirely elementwise, but splitting the kernels which are will always win a little



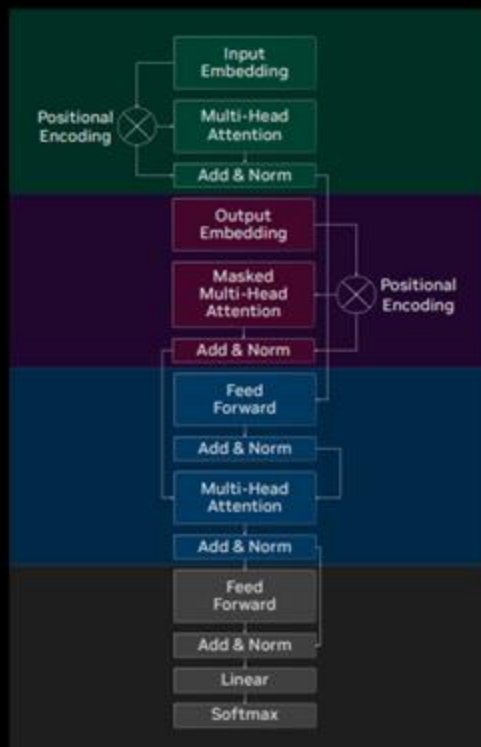
```
void daxpy(int n, double alpha, double *x, double *y)
{
    for( i = 0; i < n; i++ )
    {
        y[i] = alpha * x[i] + y[i];
    }
}
```

Elementwise "ax plus y" vector scale-and-addition

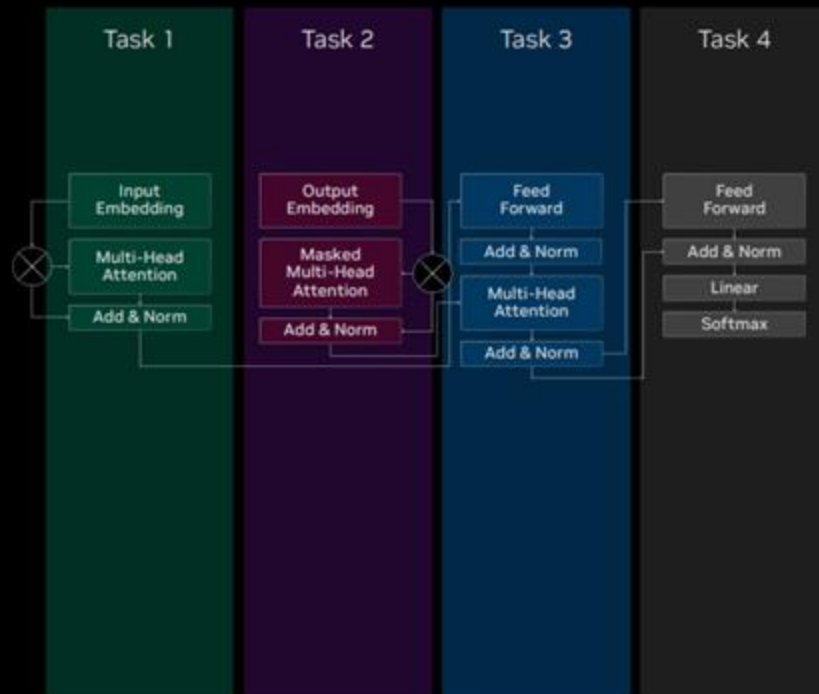
Model Parallelism: Task Parallelism For Complex Workflows



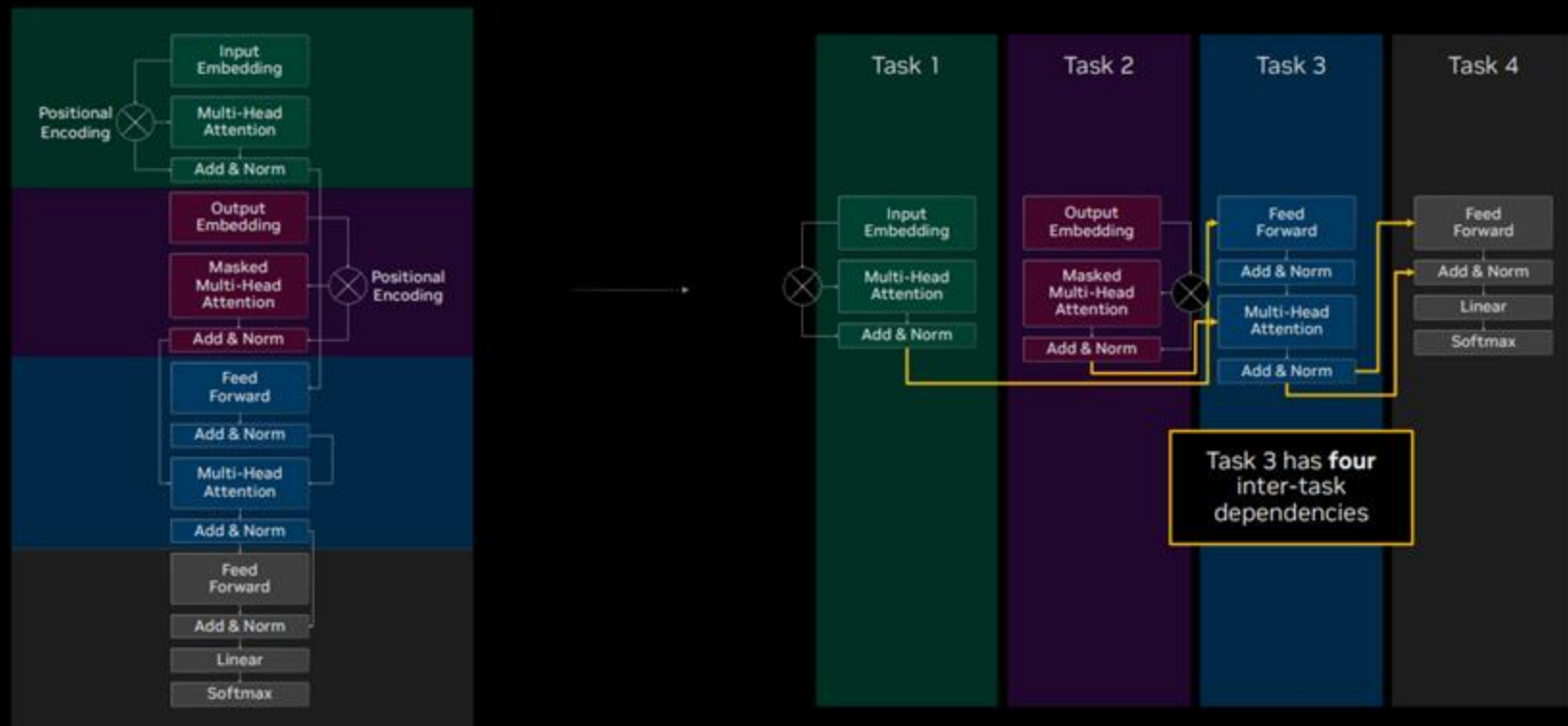
Naïve split of tasks to span GPUs



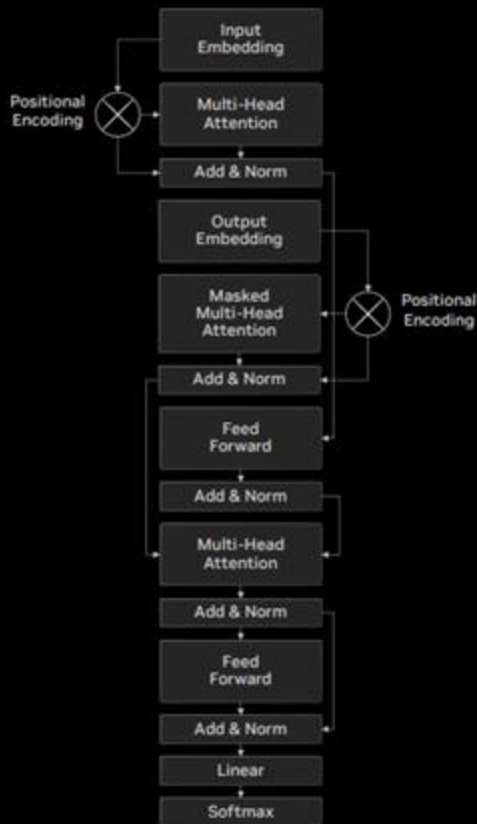
→

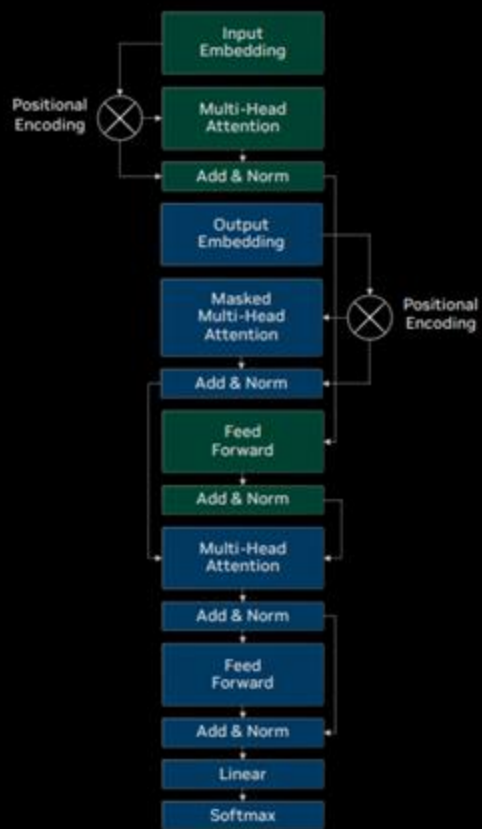


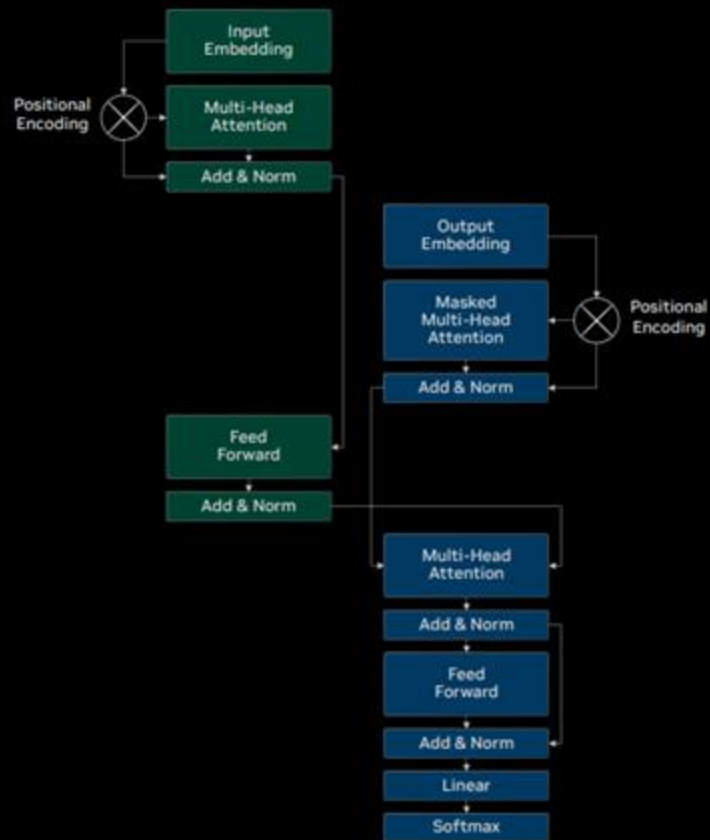
Naïve split of tasks to span GPUs

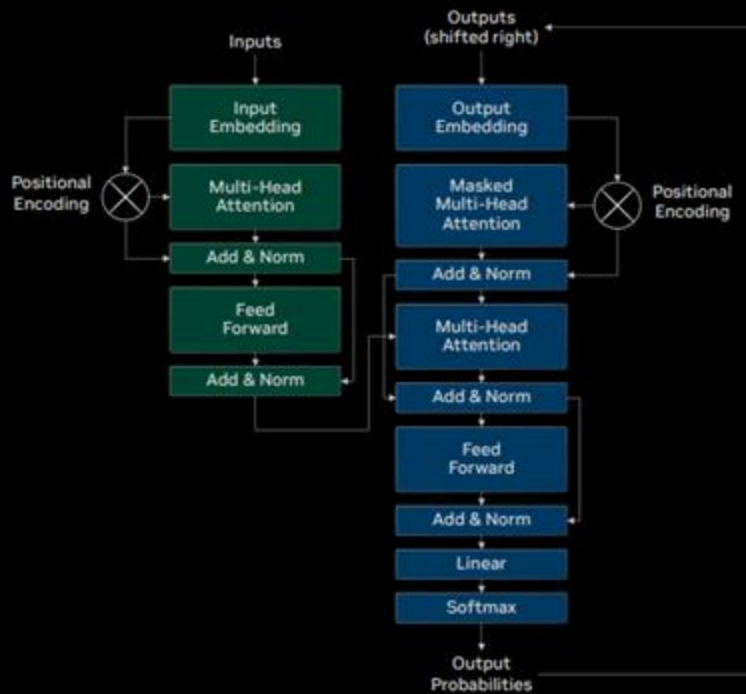


Don't Be Naïve: Minimise Inter-Task Dependencies

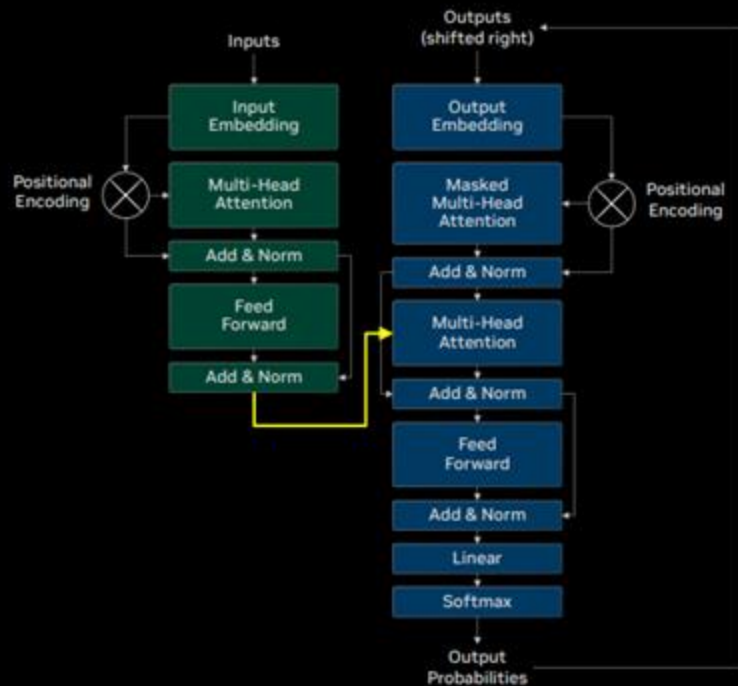




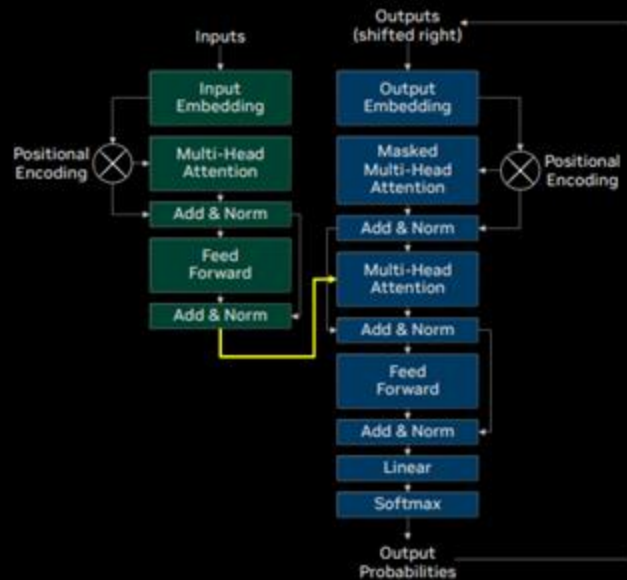




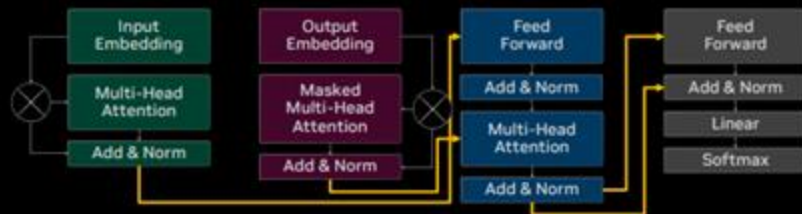
Unbalanced Tasks Are Often Much Cheaper Than Over-Synchronization



In Other Words, Split The Model To Minimise Inter-Task Sync

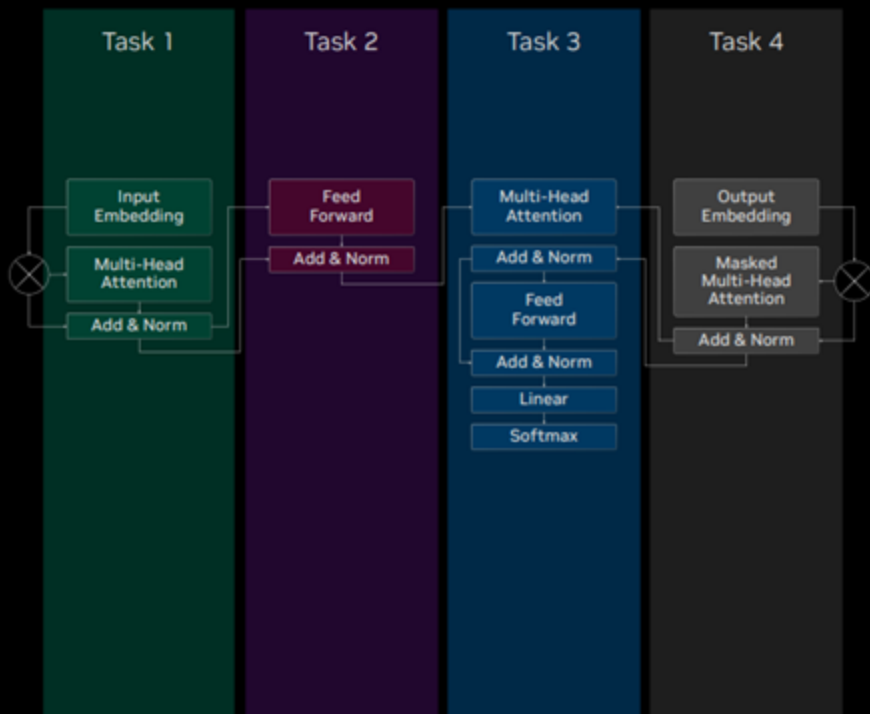
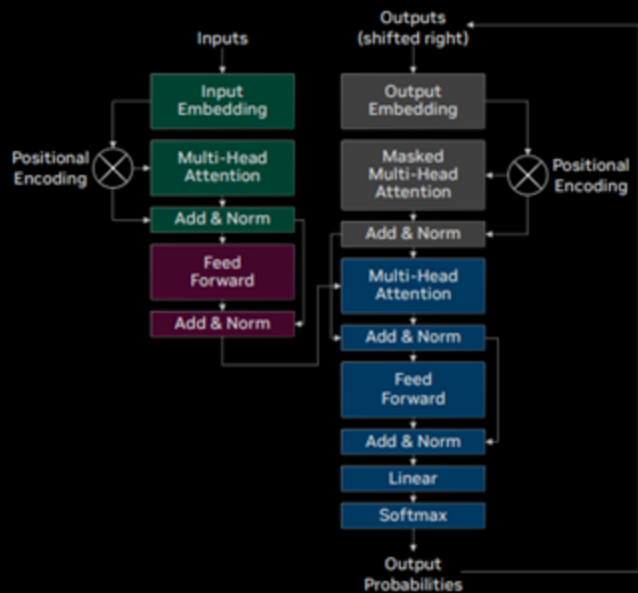


Producer/Consumer split: one dependency



Naïve split: four dependencies in/out of one task

And If You Need To Split More, Do It Hierarchically



What's Next?

The recent announcement of NVIDIA's Blackwell architecture brings significant advancements that will shape the future of HPC programming.

Architectural Innovations:

The Blackwell architecture, officially announced at NVIDIA's GTC 2024 keynote, represents a major leap forward in GPU technology for HPC and AI applications. Some key architectural innovations include:

- Increased Transistor Count: Blackwell GPUs pack 208 billion transistors, a significant increase from Hopper's 80 billion, allowing for more complex computations
- Enhanced Memory: Blackwell features HBM3e memory, providing higher bandwidth and capacity crucial for large-scale HPC simulations
- Improved Interconnect: The fifth-generation NVLink offers 1.8TB/s bidirectional throughput per GPU, enabling seamless communication among up to 576 GPUs